

Department of Computer Science

University of Engineering and Technology Lahore Pakistan

هم نوا

Hamnawa

Accessible Document Reader for Visually Impaired Students

Project Report

Subject: Software Engineering **Group No.:** 2

Submitted by:

Muhammad Sarim	2025-CS-191
Abdullah Akbar	2025-CS-212
Muhammad Junaid Amir	2025-CS-193
Muhammad Ahmad Chaudhry	2025-CS-184

Submitted to:

Dr. Maimoona Salam

November 6, 2026

Document Control

Project	Hamnawa (ہم نوا) — Accessible Document Reader for Visually Impaired Students
Document	Project Report: Proposal, Key Requirements, System Architecture, Design Architecture and Object-Oriented Design Model, version 0.1 (draft)
Course	Software Engineering, Department of Computer Science, UET Lahore
Group	Group 2 (Muhammad Sarim 2025-CS-191, Abdullah Akbar 2025-CS-212, Muhammad Junaid Amir 2025-CS-193, Muhammad Ahmad Chaudhry 2025-CS-184)
Supervisor	Dr. Maimoona Salam
Status	Draft for supervisor review

Contents at a Glance

Part	Title	Question it answers
I	Project Proposal	What are we building, for whom, and how will we manage it?
II	Key Requirements	What must the system do, and how well?
III	System Architecture	What are the parts of the system, and how do they fit together?
IV	Design Architecture	Monolith or microservices, and why?
V	OOP Design Model	Which classes and patterns implement the architecture, and why?

About this report. This document combines and condenses five standalone documents of the Hamnawa project into one. Only the essential parts of each are kept. Requirement, use-case and component identifiers (for example FR-18, UC-02) are the same as in the standalone documents, which hold the complete lists, the remaining CRC cards, the data dictionary and the detailed use-case flows.

Contents

Part I: Project Proposal **5**

1 Introduction **5**

2 Objectives and Scope **6**

3 Stakeholders and Proposed Solution **7**

4 Process Model, Feasibility and Risk **8**

5 Project Plan and Quality Assurance **9**

Part II: Key Requirements **11**

6 Requirements Overview **11**

7 Functional Requirements (Selected) **12**

8 Non-Functional Requirements (Selected) **15**

Part III: System Architecture **17**

9 Architectural Drivers and Style **17**

10 Architectural Views **18**

11 Use-Case Model **22**

12 Class-Responsibility-Collaborator (CRC) Cards **24**

13 Data Model **25**

14 Quality Attributes and Architectural Tactics **26**

Part IV: Design Architecture: Monolith or Microservices **28**

15 The Decision **28**

16 Options and Evaluation **29**

17 Why Not Microservices, and What It Costs **32**

18 Evolution Path and Decision Record **34**

Part V: Object-Oriented Design Model **35**

19 The Design Model and Why We Chose It **35**

20 Concepts and Principles Applied **36**

21 Static Model (Class View) **37**

22 Dynamic Model (Behaviour) **41**

23 Conclusion and Next Steps **44**

Glossary **45**

References **46**

List of Figures

1 Draft project timeline (weeks). 10

2 Layered view. Calls go downward; events travel upward. The browser's speech, storage and worker features are reached only through layer 4. 18

3 Processing view: a document flows left to right; any failure is routed to the Announcer so the user is always told what happened. 20

4 System-level use-case diagram. UC-09 is new and derives from FR-07 (erase all stored data). 23

5 Entity-relationship diagram of Hamnawa. PK = primary key, FK = foreign key (italic). Cardinalities are shown next to the entity they apply to. 26

6 Option A, shown for comparison only: 1 gateway + 5 services + 5 data stores = 11 parts to build, deploy, secure and keep running. The user's documents would travel across the internet to be processed. 30

7 Option C, the chosen design: a single bundle with four internal layers and one background worker. Nothing of ours runs on a server. 30

8 Weighted score of each option (out of 5). 31

9 Notation used in the class diagrams. 37

10 Class diagram of the document model. `cue()` is overridden in each subclass (polymorphism). 38

11 Interfaces at the boundaries. Dashed arrows from left show use; dashed triangles from right show implementation. Controllers never mention a browser feature. . . 39

12 Observer pattern. Controllers publish events; the interface and the services subscribe. 41

13 State diagram of `PlaybackController`. A speech failure moves Playing to Paused and announces the problem (NFR-R3). 42

14 Sequence diagram: opening a document and reading aloud. Solid arrows are calls, dashed arrows are returns or events. Each call goes through an interface, so any participant can be a fake in tests. 43

PART I

Project Proposal

1 Introduction

1.1 Background

Almost everything a university student learns from arrives as a document: lecture slides, handouts, past papers and textbooks. For most students, opening a PDF is effortless. For a student who is blind or has low vision, the same file can be a wall. Screen readers can read plain text, but many course files are not plain text: slides have no clear reading order, multi-column layouts are read line by line across both columns, headers and page numbers interrupt every paragraph, and there is usually no quick way to jump to "the next section". As a result, visually impaired students often depend on classmates, volunteers or manual re-typing, which costs them time, privacy and independence.

1.2 Problem Statement

Existing readers either read a document *linearly and literally*, or require the document to have been prepared accessibly in advance. Students cannot control how teachers prepare their files. There is no simple, free tool that takes an ordinary course document, recovers its *structure* (headings, lists, slides), and lets the student *navigate it by voice and keyboard* the way a sighted reader skims a page.

1.3 The Name and the Vision

Hamnawa (هم نوا) is an Urdu and Persian word meaning *one who shares your voice*: a companion, an ally. The software does not replace the student's effort; it stands beside the student and gives study material a voice they can control.

Every document a classmate can open, a visually impaired student can open too — and move through just as freely.

2 Objectives and Scope

2.1 Objectives

ID	Objective	How we will measure it
O1	Extract readable, correctly ordered text from text-based English PDF and PowerPoint (PPTX) files.	Reading-order accuracy on a test set of at least 20 course documents.
O2	Recover document structure (headings, paragraphs, lists, slides) as navigable sections.	Percentage of headings correctly detected.
O3	Read aloud with adjustable speed, voice, pause, resume and repeat.	All playback controls pass system tests.
O4	Allow complete use without a mouse, by keyboard and a small set of voice commands.	Every core task completed by a tester using keyboard only.
O5	Meet accessibility guidance (WCAG 2.2 AA principles) and work with common screen readers.	Accessibility checklist and manual NVDA testing.
O6	Follow a documented software engineering process and deliver the full set of engineering artifacts.	Artifacts reviewed against a checklist.

2.2 Scope

In scope	Out of scope (future work)
A web app that accepts text-based PDF and PPTX files up to 20 MB and 100 pages or slides.	Optical character recognition (OCR) for scanned documents.
Structure recovery: headings, paragraphs, lists, slides, outline.	Automatic description of images, diagrams and equations.
A reader with text-to-speech, speed and voice control, pause, resume, repeat and skip.	Urdu-language documents and Urdu voice commands.
Navigation by section, heading, slide or page, by keyboard and simple voice commands.	Native mobile apps and account-based cloud storage.
Bookmarks, resume, and an accessible interface (labels, focus order, high contrast).	—

Assumptions and constraints. Users have a modern desktop browser (speech tools differ between browsers); all tools are free and open source, so the budget is effectively zero; the

project must finish within the semester.

3 Stakeholders and Proposed Solution

3.1 Stakeholders

Stakeholder	Interest in the system	Involvement
Visually impaired and low-vision students	Primary users; need independent access to course material, large text and contrast.	Interviews, usability testing
Teachers and supervisors	Their ordinary files should just work; they approve the project.	Reviews, feedback
Student-affairs / disability office	Know real needs and existing support.	Requirements input
Development team	Deliver a correct, tested, documented product.	Build and test

3.2 A Day with Hamnawa

Ayesha opens Hamnawa and uploads this week's lecture slides. The system announces: *"Operating Systems, Lecture 5. 24 slides, 6 sections."* She says *"next section"*, hears the section titles, picks section three, slows the voice a little and listens. Halfway through she says *"bookmark"* and closes the laptop. The next morning the system says: *"Welcome back. Resume from slide 14?"*

3.3 Features by Priority (MoSCoW)

Priority	Features
Must have	Upload PDF/PPTX; text extraction; structure recovery; text-to-speech with speed control; pause, resume, repeat; keyboard navigation by heading and slide.
Should have	Voice commands; bookmarks and resume; high-contrast and large-text mode.
Could have	Reading highlight for low-vision users; reading history; search.
Won't have (this time)	OCR, image and equation description, Urdu support, mobile app.

3.4 What Makes Hamnawa Different

- **Structure-aware:** it understands sections and slides and lets the user move between them, instead of reading top to bottom.
- **Works with ordinary files:** teachers need not prepare special accessible versions.
- **Hands-free by design:** every task can be done without a mouse and without seeing the screen.
- **Free and lightweight:** runs in a browser; no paid services.

4 Process Model, Feasibility and Risk

4.1 Process Model

We follow the **incremental process model** (Pressman and Maxim), applying the framework activities (communication, planning, modelling, construction, deployment) in each increment. Requirements are partly unknown until we talk to real users, each increment gives something that works and can be tested early, and if time runs short we still have a working core product.

Increment	Content	Outcome
1. Core reader	Upload a text-based PDF, extract text, read aloud with basic controls.	Working end-to-end skeleton
2. Structure	Heading, paragraph and slide detection; outline; keyboard navigation; PPTX support.	Structure-aware navigation
3. Voice and memory	Voice commands, bookmarks, resume, display modes.	Hands-free, persistent experience
4. Polish and verify	Accessibility audit, user testing, bug fixing, documentation.	Release candidate

4.2 Feasibility

Dimension	Assessment
Technical	Feasible. PDF and PPTX text extraction and browser speech synthesis use mature, free libraries. The hardest part, reading order and structure, is bounded by our scope.
Economic	Feasible. Estimated cost is Rs. 0: free tools, free static hosting or local demo, built-in browser speech, free screen reader (NVDA) for testing.
Operational	Feasible. Runs in a browser, needs no installation, and fits how students already use screen readers.

Dimension	Assessment
Schedule	Feasible if scope is respected; incremental delivery protects a working core.
Legal and ethical	Low risk. Only user-supplied files are processed; library licences are checked; user testing follows consent practice.

4.3 Main Risks

ID	Risk	Prob.	Impact	Mitigation
R1	PDF layouts vary, so reading order is wrong.	High	High	Restrict scope to text-based files; build a test corpus; warn on risky files.
R2	Browser speech recognition is inaccurate or unavailable.	Medium	Medium	Keyboard is the primary path; voice is a "should have".
R3	No visually impaired user available for testing.	Medium	High	Contact accessibility staff early; fall back to screen-off NVDA testing by the team.
R4	Scope creep (OCR, Urdu, images).	High	High	Fixed scope statement; change requests reviewed by the team.
R5	Free hosting limits or outage during the demo.	Low	High	Local demo build and recorded backup demo.
R6	Open-source library bugs or licence issues.	Low	Medium	Evaluate early; keep extraction behind a replaceable interface.

5 Project Plan and Quality Assurance

5.1 Schedule

The draft schedule below is in weeks and will be fitted to the real semester calendar. **[replace with actual deadline]**

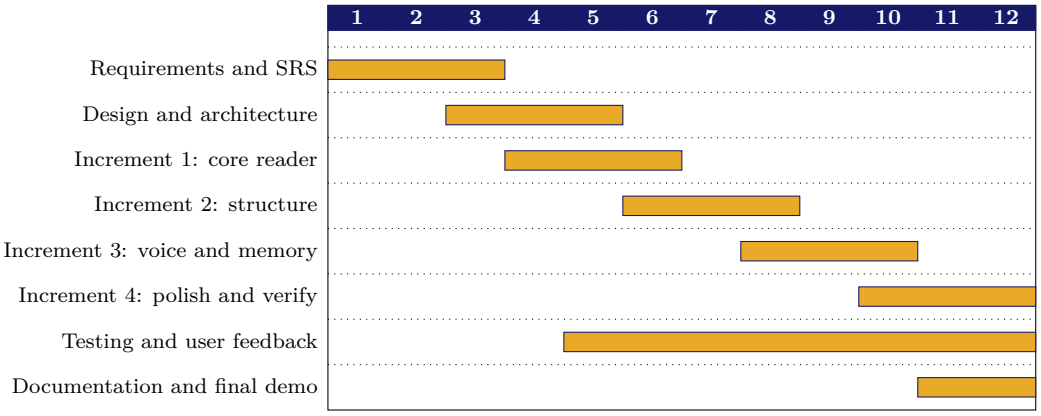


Figure 1: Draft project timeline (weeks).

5.2 Team Organisation

Role	Responsibility	Assigned to
Project manager	Plan, schedule, risk tracking, contact with the supervisor.	[name]
Requirements and documentation lead	Interviews, requirements, use cases, user manual.	[name]
Design and architecture lead	Architecture, UML models, design documents.	[name]
Implementation lead	Core code, code reviews, repository health.	[name]
Quality and test lead	Test plan, test cases, accessibility testing, metrics.	[name]

With four members, some roles are shared; every member takes part in both development and testing.

5.3 Quality Assurance

- **Reviews:** peer review of requirements, design and key code before each increment ends.
- **Testing levels:** unit tests (extraction, structure, command parsing); integration tests of the full pipeline on 20 or more course documents; system and acceptance tests with every requirement traced to a test case.
- **Accessibility testing:** keyboard-only walkthroughs, NVDA checks and a WCAG-based checklist.
- **Metrics and configuration management:** reading-order accuracy, heading-detection rate, command response time, defects per increment; Git with a branch per increment and reviewed merges.

PART II

Key Requirements

6 Requirements Overview

6.1 Conventions

- **IDs.** Functional requirements are `FR-nn`; non-functional requirements are `NFR-xn`, where the letter is the quality group (U usability, A accessibility, P performance, R reliability, S security and privacy, C compatibility, M maintainability).
- **Priority (Pri.).** M = Must have, S = Should have. **Verification (Ver.).** T = Test, D = Demonstration, I = Inspection, A = Analysis.
- **Objective (Obj.).** The objective O1 to O5 of Part I that the requirement supports.
- "Shall" marks a mandatory statement. Every requirement can be tested on its own.

Functional group	Shown here	In full set
Document Management	4	7
Extraction and Structure Recovery	7	10
Playback	5	10
Navigation	4	7
Voice Control	3	5
Bookmarks and Resume	2	4
Settings, Display and Help	3	4
Total functional	**28**	**47**
Total non-functional	**14**	**28**

7 Functional Requirements (Selected)

All *Must* requirements are listed, plus the Should requirements that define the voice, bookmark and display features.

7.1 Document Management

How a document enters the system and how the user is told about it.

ID	Requirement	Pri.	Ver.	Obj.
FR-01	The system shall let the user select a PDF or PPTX file using a keyboard-operable file selector.	M	T	O4
FR-02	The system shall validate file type, file size (maximum 20 MB) and length (maximum 100 pages or slides) before processing, and report the result by speech and text.	M	T	O1
FR-03	The system shall detect files it cannot read (scanned or image-only, password-protected, corrupted, unsupported type) and state the specific reason and a suggested action.	M	T	O1
FR-05	When a document opens, the system shall announce its title (if found), number of pages or slides, and number of top-level sections.	M	D	O2

7.2 Extraction and Structure Recovery

Turning an ordinary file into a correctly ordered, structured document model.

ID	Requirement	Pri.	Ver.	Obj.
FR-08	The system shall extract text from text-based PDF files in correct reading order.	M	T	O1
FR-09	The system shall extract text from PPTX files: slide title, body text in order, and (user-selectable) speaker notes.	M	T	O1
FR-11	The system shall detect and exclude repeated headers, footers and page numbers from the spoken text.	M	T	O1
FR-12	The system shall identify headings and their levels using font size, weight and numbering patterns.	M	T	O2
FR-13	The system shall identify paragraphs and bulleted or numbered lists, and announce lists with their item count.	M	T	O2
FR-14	The system shall identify slide boundaries and slide titles; a slide without a title shall be named “Slide N”.	M	T	O2
FR-15	The system shall build a hierarchical outline (sections, subsections, slides) that serves as the navigation model.	M	T	O2

7.3 Playback

Speaking the document and giving the user control of the speech.

ID	Requirement	Pri.	Ver.	Obj.
FR-18	The system shall read the document aloud from the current position using text-to-speech.	M	T	O3
FR-19	The system shall provide pause, resume and stop of speech.	M	T	O3
FR-20	The system shall repeat the current sentence or paragraph on request.	M	T	O3
FR-21	The system shall skip to the next or previous sentence or paragraph on request.	M	T	O3
FR-22	The system shall let the user set speech rate between 0.5× and 3.0× in steps of 0.25×, taking effect within 1 second.	M	T	O3

7.4 Navigation

Moving around a document by structure, not just top to bottom.

ID	Requirement	Pri.	Ver.	Obj.
FR-28	The system shall move to the next or previous section on request.	M	T	O4
FR-30	The system shall go directly to a given slide or page number.	M	T	O4
FR-31	The system shall provide an outline list that can be browsed with the arrow keys, with Enter jumping to the selected entry.	M	T	O4
FR-33	The system shall jump to the beginning or end of the document on request.	M	T	O4

7.5 Voice Control

Hands-free operation using a small, fixed set of spoken commands.

ID	Requirement	Pri.	Ver.	Obj.
FR-35	The system shall recognise a fixed set of English voice commands (for example play, pause, repeat, next section, go to slide n, bookmark, help) and perform the matching action.	S	T	O4
FR-36	The microphone shall be off by default and shall be switched on and off by the user with a keyboard shortcut or a spoken command.	S	T	O4
FR-39	If speech recognition is unavailable or permission is denied, the system shall say so and continue to work fully by keyboard.	S	T	O4

7.6 Bookmarks and Resume

Remembering where the user was and what they marked.

ID	Requirement	Pri.	Ver.	Obj.
FR-40	The system shall add a bookmark at the current position, with an optional typed or spoken label.	S	T	O4
FR-43	When the same document is opened again (identified by a content fingerprint), the system shall offer to resume from the saved position.	S	T	O4

7.7 Settings, Display and Help

Making the interface comfortable for low-vision users and easy to learn.

ID	Requirement	Pri.	Ver.	Obj.
FR-44	The system shall offer a high-contrast display mode and text size adjustable from 100% to 200%.	S	T	O5
FR-46	The system shall provide a help command that reads out the available keyboard shortcuts and voice commands.	M	T	O5
FR-47	Every status and error message shall be delivered both as speech and as on-screen text.	M	I	O5

8 Non-Functional Requirements (Selected)

Each requirement has a *fit criterion*, a measurable statement that tells the tester when it is met. Numeric targets marked "target" are first estimates, to be confirmed by measurement in the first increment.

ID	Requirement	Fit criterion (measure)	Ver.
NFR-U1	A new user shall be able to open a document and start listening using the keyboard only.	At most 3 steps from page load to speech starting.	T
NFR-U2	Every function shall be reachable without a mouse.	100% of functional requirements operable by keyboard; checked by a keyboard-only walkthrough.	T
NFR-A1	The interface shall conform to WCAG 2.2 level AA principles.	Zero open failures of AA criteria in the final accessibility checklist.	I
NFR-A5	The product shall work with the NVDA screen reader.	All core tasks completed with NVDA running.	T
NFR-P1	Reading shall begin promptly after a document is opened.	Speech starts within 5 s for a 50-page document (target, to be validated in Increment 1).	T
NFR-P2	Keyboard commands shall feel immediate.	Response (speech or focus change) within 200 ms for 95% of key presses.	T
NFR-P4	Document processing shall complete in reasonable time.	At most 20 s for a 100-page document on a typical student laptop (target).	T

ID	Requirement	Fit criterion (measure)	Ver.
NFR-R1	A bad or unsupported file shall never crash the application.	All files in the negative test set produce a spoken error, with no crash.	T
NFR-R3	If speech output fails, the user shall be told and the text remain available.	Failure simulated; message shown and text readable.	T
NFR-S1	Documents shall be processed on the user's device where technically possible; any server copy shall be deleted at session end.	No document stored after the session; verified by design review and test.	I
NFR-S2	Bookmarks, positions and settings shall be stored locally only.	No personal data sent to the project's servers.	I
NFR-C1	The product shall run in current desktop Chrome and Edge on Windows 10 and 11.	Core tasks pass on the latest two versions of each.	T
NFR-M1	The code shall be organised in modules: extraction, structuring, playback, navigation, interface.	Each module has a documented interface and can be tested alone.	I
NFR-M2	Core logic shall be covered by automated tests.	At least 70% statement coverage for extraction and structuring modules.	A

8.1 Validation Status

These requirements come from the proposal, accessibility guidance and the team's analysis. They have *not yet* been validated with real users: interviews with at least one visually impaired or low-vision student, and a formal team review, are planned (Increment 1). The file limits (FR-02), keyboard shortcuts and performance targets will be confirmed then.

PART III

System Architecture

9 Architectural Drivers and Style

9.1 Drivers

A small number of requirements shape the architecture and are hard to change later. Every major decision in Section 10.5 answers at least one of them.

Driver	Why it matters	Requirements
Accessibility first	The user cannot see the screen; every function must work by keyboard, speech and a screen reader. The interface must be semantic HTML, not a drawing surface.	NFR-A1 to A6, NFR-U1, U2, FR-27, FR-47
Privacy and zero cost	Course documents may be personal; there is no budget for servers. Processing on the user's device avoids both problems.	NFR-S1, S2, cost constraint
Responsiveness	Parsing a large file must not freeze the keyboard or the speech. Commands must feel immediate.	NFR-P1 to P4
Robustness	Real course files are messy. A bad file must produce a spoken explanation, never a crash.	NFR-R1, NFR-S4, FR-03
Dependence on browser speech	Speech output and recognition come from the browser and differ between browsers. They may fail or be missing.	NFR-C1 to C3, FR-39, NFR-R3
Incremental delivery	The team delivers in four increments; each must run. Modules must be addable without rewriting earlier ones.	Process model, NFR-M1
Testability	Core logic must be testable without a browser or a microphone.	NFR-M1, M2

9.2 Architectural Style

Hamnawa combines three well-known styles:

1. **Layered architecture.** Four layers (presentation, application, domain and processing, infrastructure). A layer may use only the layers below it, which keeps the interface separate from logic and makes the logic testable.
2. **Pipeline (pipes and filters).** A document flows through validate, extract, clean and structure, and build model. Each step has one job and can be tested and replaced independently.
3. **Adapters at the edges.** Everything that depends on the browser (speech output, speech input, storage) is hidden behind a small interface, so a missing browser feature affects one class only.

Components talk by direct calls downward and by *events* upward (for example `documentReady`, `positionChanged`), so lower layers never depend on the interface.

10 Architectural Views

10.1 Layered View

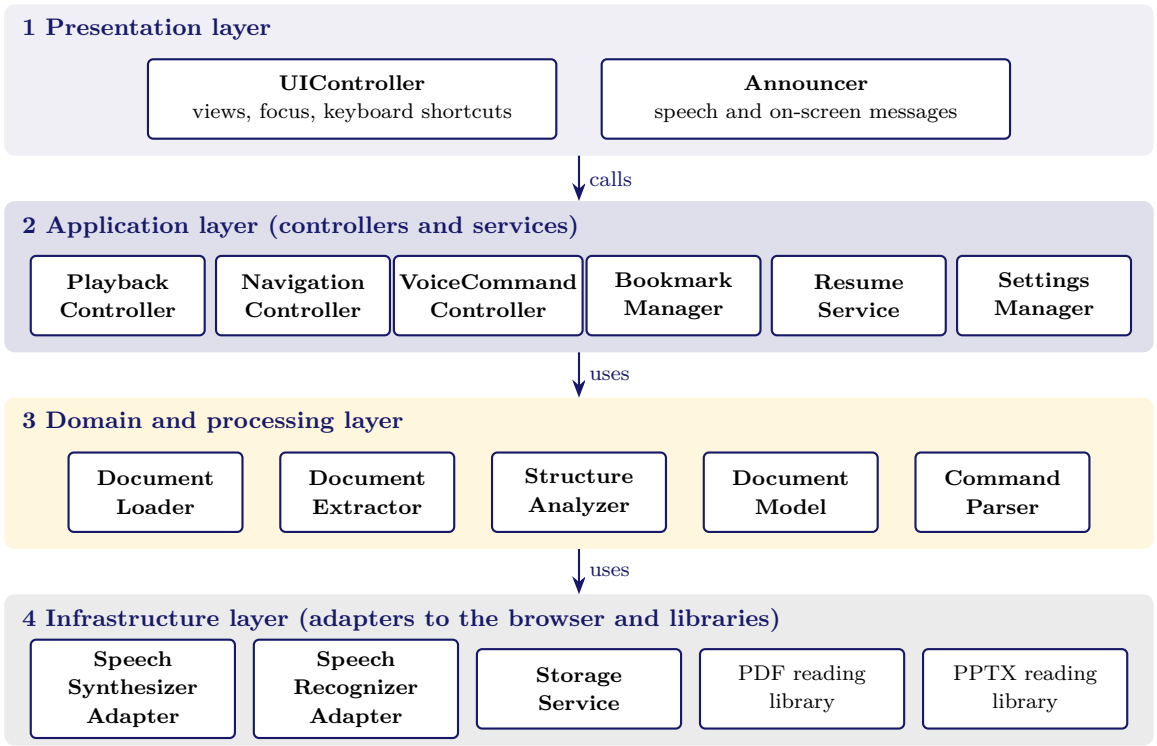


Figure 2: Layered view. Calls go downward; events travel upward. The browser's speech, storage and worker features are reached only through layer 4.

10.2 Component Responsibilities

The table summarises each of the 16 components, its layer and the increment in which it is first delivered. The four most important CRC cards are in Section 12.

Component	Layer	Summary	Inc.
`UIController`	Presentation	Renders the three regions, manages focus and keyboard shortcuts, high contrast and text size, help.	1
`Announcer`	Presentation	Delivers every status and error message as speech and as on-screen text.	1
`PlaybackController`	Application	Controls reading: play, pause, repeat, skip, speed, voice, structural cues, continuous reading.	1
`NavigationController`	Application	Moves through the document by section, heading level, slide or page; where am I; search.	2
`VoiceCommandController`	Application	Switches the microphone on and off, listens, and turns recognised phrases into actions.	3
`BookmarkManager`	Application	Adds, lists, deletes and jumps to bookmarks of the open document.	3
`ResumeService`	Application	Saves the last reading position, offers to resume, and keeps the recent-documents list.	3
`SettingsManager`	Application	Loads, saves and applies user settings; erases all stored data on request.	3
`DocumentLoader`	Domain and proc.	Accepts a file, validates it, computes its fingerprint and runs the processing pipeline.	1
`DocumentExtractor`	Domain and proc.	Reads text, order and style information from PDF and PPTX files (one subclass for each).	1–2
`StructureAnalyzer`	Domain and proc.	Cleans the raw text and recovers headings, lists, paragraphs, slides and the outline.	2
`DocumentModel`	Domain and proc.	Holds the outline and content blocks and answers position and navigation queries.	1–2

Component	Layer	Summary	Inc.
`CommandParser`	Domain and proc.	Turns a recognised phrase into a command with parameters, or reports it as unknown.	3
`SpeechSynthesizerAdapter`	Infrastructure	Wraps the browser's text-to-speech and hides its differences and failures.	1
`SpeechRecognizerAdapter`	Infrastructure	Wraps the browser's speech recognition and the microphone permission.	3
`StorageService`	Infrastructure	Stores and retrieves records in the browser's local storage; reports quota and access errors.	3

10.3 Document Processing Pipeline

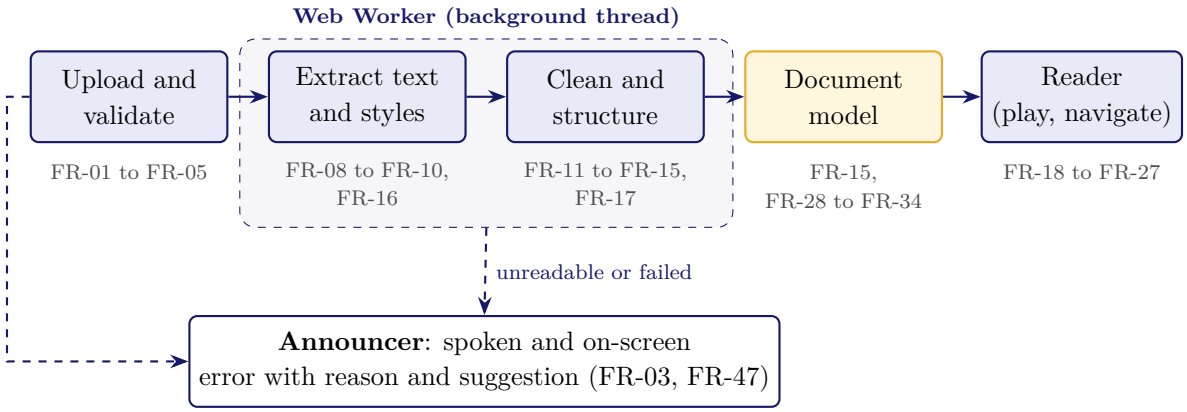


Figure 3: Processing view: a document flows left to right; any failure is routed to the Announcer so the user is always told what happened.

10.4 Technology Choices (Proposed)

All choices are free and open source and will be confirmed in the first increment (licence check: NFR-L1).

Concern	Choice	Reason
Language	JavaScript (or TypeScript)	Runs natively in the browser; one language for the whole product.
User interface	Semantic HTML, ARIA, CSS (no canvas)	Screen readers and keyboard focus work only with real elements.
PDF reading	pdf.js	Mature open-source library that exposes text with position and font data.

Concern	Choice	Reason
PPTX reading	Unzip and parse the XML inside the file	A PPTX file is a zip of XML; slide titles and text can be read directly.
Speech output and input	Web Speech API (speech synthesis, speech recognition)	Built into Chrome and Edge, free, no server.
Background processing	Web Worker	Keeps the interface and speech responsive while a file is parsed.
Local storage	IndexedDB or localStorage	Stores bookmarks, positions and settings only on the user's device.
Document fingerprint	Web Crypto (SHA-256)	Gives a stable identity for a file.
Testing	A JavaScript test runner; NVDA; an accessibility checker	Unit tests for core logic; manual and automated accessibility checks.
Hosting and version control	Free static hosting; Git and GitHub	No server cost; history and review of changes.

10.5 Key Design Decisions

ID	Decision	Alternatives considered	Rationale and trade-off
D1	Process documents entirely in the browser.	Upload to a server and process there.	Protects privacy and costs nothing. Trade-off: limited by the user's device, so heavy work goes to a Web Worker. Revisit if Increment 1 shows poor speed.
D2	Layered style with a pipeline for documents.	Single large script; model-view-controller only.	Clear boundaries, easy unit tests, parallel work by the team. Trade-off: more files and interfaces.
D3	Wrap speech and storage in adapter classes.	Call browser APIs directly everywhere.	Isolates browser differences and failures (FR-39, NFR-R3) and lets tests use fake adapters.
D4	One format-independent document model.	Separate logic for PDF and PPTX.	Playback, navigation and bookmarks work the same for both formats; only extractors differ.

ID	Decision	Alternatives considered	Rationale and trade-off
D5	Run extraction and structuring in a Web Worker.	Run on the main thread.	Keeps keys and speech responsive (NFR-P2). Trade-off: messages between threads must be designed.
D6	Identify documents by content fingerprint.	Use the file name.	Bookmarks survive renaming and copying.
D7	Components communicate through events upward.	Direct calls both ways.	Lower layers stay independent of the interface; fewer circular dependencies.
D8	Sentence-by-sentence speech queue.	Hand the whole text to the speech engine.	Gives exact position for repeat, skip and resume, and avoids known browser problems with very long utterances.

11 Use-Case Model

11.1 Actors

The **Student** is the primary actor: a blind or low-vision student who studies from course documents. The **browser speech services** are a supporting system actor. Teachers are not actors; they only supply ordinary documents.

11.2 Use-Case Catalogue

ID	Name	Goal	Requirements	Inc.
UC-01	Open a document	Load a PDF or PPTX and learn what it contains.	FR-01 to FR-05, FR-08 to FR-17	1–2
UC-02	Listen to the document	Hear the content with full playback control.	FR-18 to FR-27	1
UC-03	Navigate by structure	Jump between sections, slides and pages.	FR-28 to FR-34	2
UC-04	Control by voice	Operate the reader with spoken commands.	FR-35 to FR-39	3

ID	Name	Goal	Requirements	Inc.
UC-05	Manage bookmarks	Mark, list and revisit important places.	FR-40, FR-41	3
UC-06	Resume reading	Continue from the last saved position.	FR-42, FR-43	3
UC-07	Adjust settings	Change speed, voice, contrast and text size.	FR-22, FR-23, FR-44, FR-45	3
UC-08	Get help	Hear available shortcuts and commands.	FR-46	1
UC-09	Erase stored data	Remove all bookmarks, positions, history and settings.	FR-07	3

11.3 Use-Case Diagram

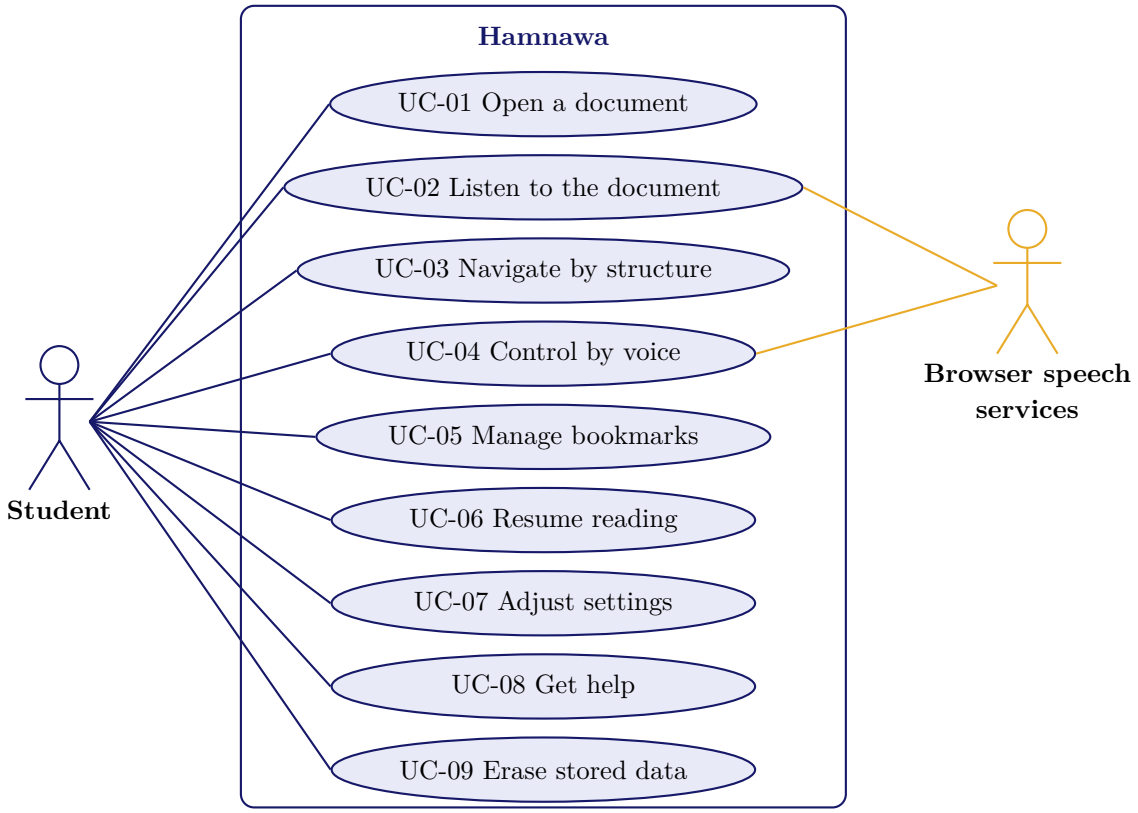


Figure 4: System-level use-case diagram. UC-09 is new and derives from FR-07 (erase all stored data).

12 Class-Responsibility-Collaborator (CRC) Cards

A CRC card lists a class, what it is responsible for, and which other classes it works with. It is a quick way to find the responsibilities of a design before drawing detailed diagrams. Four core classes are shown, one from each layer that carries the main behaviour; the other twelve are in the standalone System Architecture Document.

PlaybackController — Application layer

Responsibilities	Collaborators
• Start, pause, resume and stop reading from the current position • Repeat the current sentence or paragraph; skip sentences and paragraphs • Apply speech rate, voice and volume from the settings • Announce structural cues (heading, list, slide) before the element • Continue into the next section or stop, as set; announce the end of the document • Publish position changes as events	• DocumentModel • SpeechSynthesizerAdapter • SettingsManager • ResumeService • Announcer

Satisfies: FR-18 to FR-25, NFR-P2

DocumentLoader — Domain and processing layer

Responsibilities	Collaborators
• Accept the chosen file; check type, size and length • Compute the content fingerprint that identifies the document • Start extraction and structuring in a background worker • Detect unreadable files (scanned, protected, corrupted) and report the reason • Announce title, page or slide count and number of sections when ready	• DocumentExtractor • StructureAnalyzer • DocumentModel • ResumeService • Announcer

Satisfies: FR-01 to FR-05, FR-43, NFR-P1, NFR-P4, NFR-R1, NFR-S4

StructureAnalyzer — Domain and processing layer

Responsibilities	Collaborators
• Remove repeated headers, footers and page numbers • Repair hyphenated line breaks, ligatures and stray line breaks • Detect headings and their levels from font ranking and numbering patterns • Detect paragraphs, lists (with item count) and slide boundaries • Name untitled slides “Slide N” • Build the outline tree and hand it to the document model	• DocumentExtractor • DocumentModel

Satisfies: FR-11 to FR-15, FR-17

SpeechSynthesizerAdapter — Infrastructure layer

Responsibilities	Collaborators
• Speak one sentence at a time; report start and end of each • List available voices; set rate, voice and volume • Pause, resume and cancel speech • Detect missing or failing speech and report it	• PlaybackController • Announcer • Browser speech engine

Satisfies: FR-18, FR-22, FR-23, NFR-R3, NFR-C1 to C3

13 Data Model

Hamnawa has no server database. Data lives in two places:

- **Stored in the browser** (gold outline in the diagram): document identity, bookmarks, reading position and settings. These survive closing the browser.
- **Built in memory** (blue outline): the section tree and content blocks. They are rebuilt each time a document is opened, so they are never stored.

A document is identified by a *fingerprint* of its content, not by its file name, so bookmarks survive renaming and copying. A saved position is a pair (section order, block index), which is stable because the same document always produces the same structure.

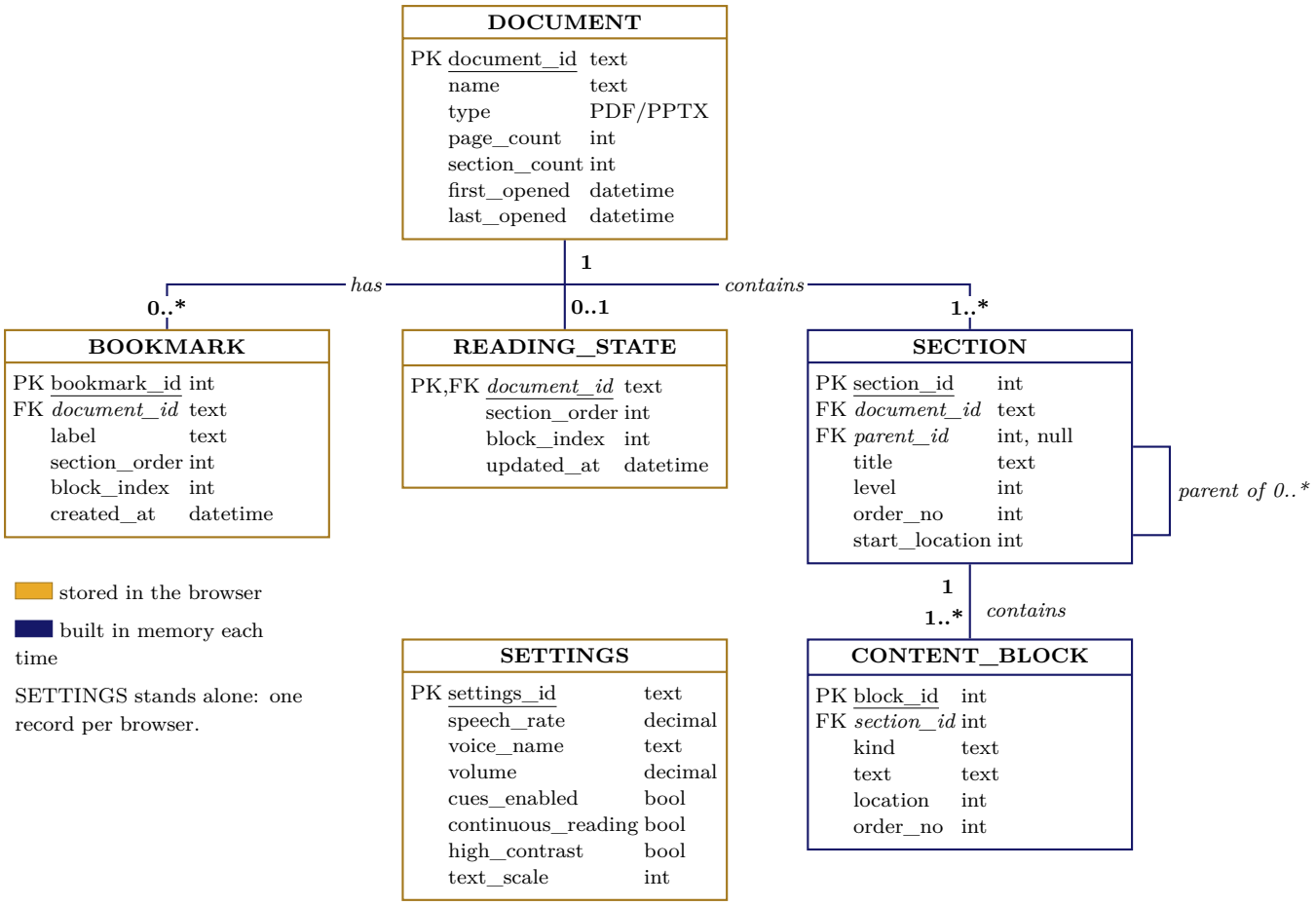


Figure 5: Entity-relationship diagram of Hamnawa. PK = primary key, FK = foreign key (italic). Cardinalities are shown next to the entity they apply to.

14 Quality Attributes and Architectural Tactics

Quality	Tactic in the architecture	Where it lives
Usability and accessibility	Semantic HTML with ARIA landmarks; one logical focus order; single-key shortcuts in the reader; every message also announced in a live region; coexistence mode for screen readers.	UIController, Announcer (NFR-A1 to A6, U1 to U4)
Performance	Extraction and structuring in a Web Worker; sentence-level speech queue; outline built once and reused; lazy loading of page content.	DocumentLoader, DocumentExtractor, PlaybackController (NFR-P1 to P4)

Quality	Tactic in the architecture	Where it lives
Reliability	Defensive parsing with a catch at the worker boundary; every failure routed to the Announcer; storage errors reported, not thrown.	DocumentLoader, StorageService, Announcer (NFR-R1 to R3, S4)
Security and privacy	No document leaves the device; data stored locally only; microphone off by default; notice before first use of recognition.	DocumentLoader, StorageService, VoiceCommandController (NFR-S1 to S3)
Compatibility	Feature detection in the adapters; clear spoken message when a feature is missing; keyboard control always available.	SpeechSynthesizerAdapter, SpeechRecognizerAdapter (NFR-C1 to C3, FR-39)
Maintainability and testability	Layered modules with documented interfaces; adapters replaceable by fakes in tests; format-independent document model.	All layers (NFR-M1 to M3)

PART IV

Design Architecture: Monolith or Microservices

15 The Decision

Decision (ADR-001). Hamnawa is a **modular monolith**: one deployable web application that runs entirely in the user's browser, organised internally into clearly separated modules. It is **not** built as microservices, and it has no backend server of our own.

15.1 What the Terms Mean

Aspect	Monolith	Microservices
Definition	One program built, tested and deployed as a single unit.	Many small programs, each owning one capability, deployed and scaled independently.
Communication	Function calls inside one process.	Network calls between processes.
Strengths	Simple to build, test, debug and deploy; fast calls; easy refactoring.	Independent scaling and release; fault isolation between teams.
Weaknesses	Can become tangled without module discipline.	Heavy operational and testing overhead; latency and network failure.
Typical fit	Small teams, one product, moderate load, tight latency.	Large organisations, many teams, very high or uneven load.

A **modular monolith** is a monolith divided into modules with strict boundaries and public interfaces: the simplicity of one deployable unit, with most of the design discipline that microservices force on a team.

15.2 Three Clarifications

- **The Web Worker is not a microservice.** It is a background thread of the same program, shipped in the same bundle.
- **The static web host is not a backend.** It hands out the application files once; it runs none of our logic and stores no user data.
- **The browser's speech engine and the vendor's recognition service are third-party services we use**, not services we build or operate.

15.3 Decision Drivers

ID	Driver	What it means for the style	Source
DR1	**Privacy of documents**	Course files should not leave the user's device; no server copy to protect or leak.	NFR-S1, S2
DR2	**Zero budget**	No paid servers, databases or cloud services.	Cost constraint
DR3	**Instant response**	Commands must act within 200 ms to 2 s; every network hop adds delay and a failure point.	NFR-P1 to P4
DR4	**Small team, one semester**	Four people, four increments; little capacity to run infrastructure.	Process model
DR5	**Testability and maintainability**	Core logic testable without a browser or microphone; clear module boundaries.	NFR-M1, M2
DR6	**Operational simplicity and reliability**	Easy to deploy, roll back and demonstrate; few parts that can fail.	NFR-R1 to R3
DR7	**Load profile**	One user per device; there is no concurrent-user load to solve.	Product scope
DR8	**Dependence on browser speech**	Speech output and recognition already live in the browser.	NFR-C1 to C3

16 Options and Evaluation

16.1 The Options

Option A: Microservices. A backend of independent services behind a gateway; the browser is a thin client (Figure 6). **Option B: Server-based monolith.** One backend application that receives documents, processes them and returns the result; it needs hosting and secure storage for user files. **Option C: Client-side modular monolith (chosen).** All logic ships as one bundle that runs in the browser; heavy parsing runs in a Web Worker (Figure 7).

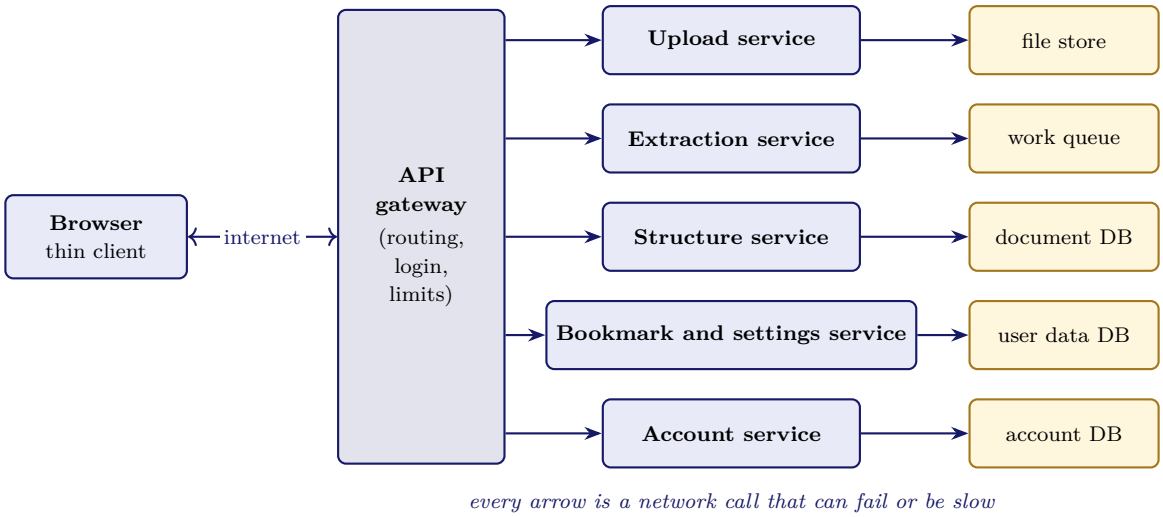


Figure 6: Option A, shown for comparison only: 1 gateway + 5 services + 5 data stores = 11 parts to build, deploy, secure and keep running. The user's documents would travel across the internet to be processed.

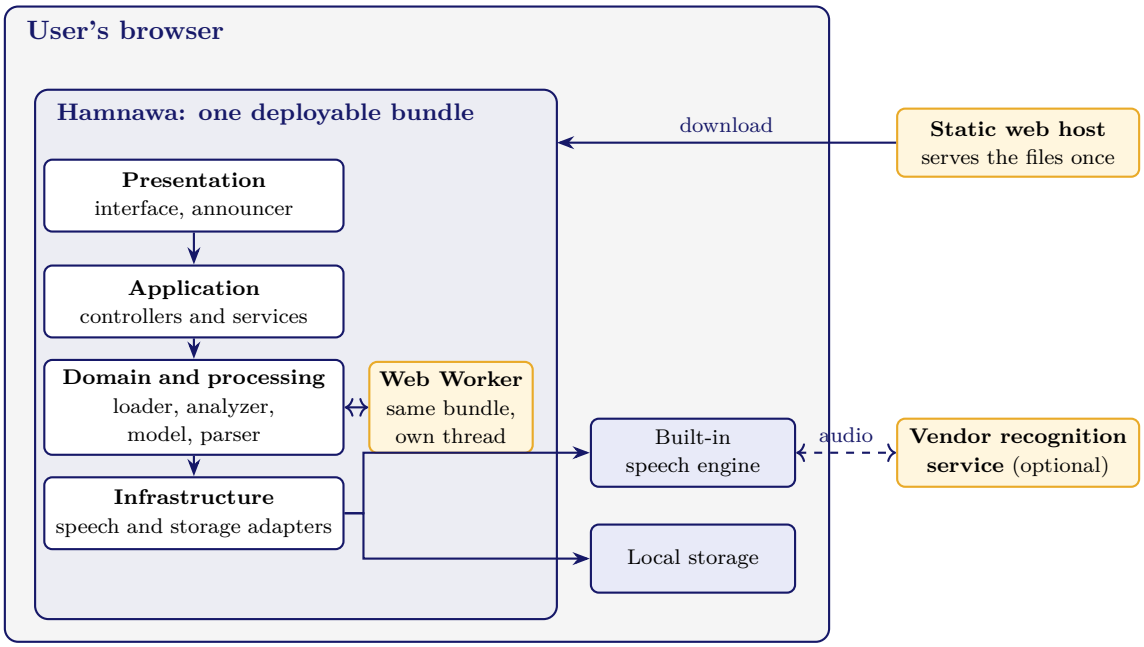


Figure 7: Option C, the chosen design: a single bundle with four internal layers and one background worker. Nothing of ours runs on a server.

16.2 Scored Comparison

Each option was scored from 1 (poor) to 5 (excellent) against the decision drivers. Weights add up to 100 and show how much each criterion matters. The scores are the team's judgement, supported by the evidence in the next subsection.

Criterion	Weight	A: Micro-services	B: Server monolith	C: Client modular monolith
Privacy of documents (DR1)	20	1	2	5
Zero cost (DR2)	15	1	2	5
Responsiveness (DR3)	15	2	3	5
Fit with team and schedule (DR4)	15	1	3	5
Testability and maintainability (DR5)	10	3	3	4
Operational simplicity (DR6)	10	1	3	5
Reliability (DR6, NFR-R)	10	2	3	4
Scalability (DR7)	5	5	3	3
Weighted score (out of 5)	100	**1.65**	**2.65**	**4.70**

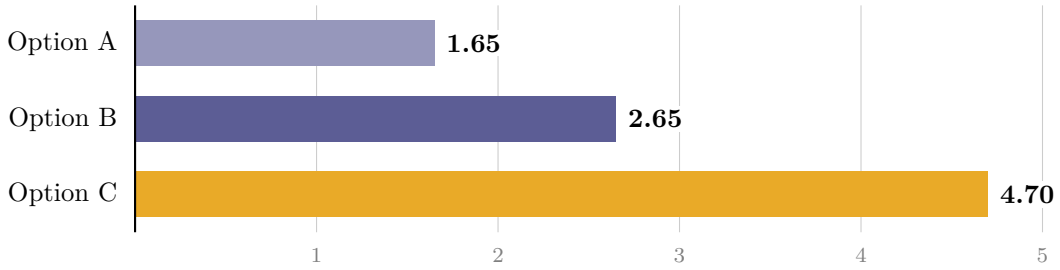


Figure 8: Weighted score of each option (out of 5).

16.3 Sensitivity Check

A decision is more convincing if it survives a change of weights. We repeated the calculation twice:

Scenario	A	B	C
Original weights	1.65	2.65	4.70
All criteria weighted equally	2.00	2.75	4.50
Privacy and cost ignored completely	2.00	3.00	4.54

Option C stays first in every scenario, so the decision does not depend on how much we value privacy and cost.

16.4 Evidence Behind the Scores

Criterion	Reasoning
Privacy	A and B send every document to a server, so we would have to secure, retain and delete user files. C never transmits the file.
Cost	A needs several servers and data stores; B needs one always-on server. C needs only free static hosting.
Responsiveness	Each request in A or B adds network time and can fail. In C, commands are function calls on the same device.
Team and schedule	A needs service contracts, a gateway, cross-service testing and deployment for each service, which would consume the time needed for the product itself.
Testability	A needs contract and end-to-end tests across the network. C keeps logic in plain modules that unit tests call directly.
Simplicity and reliability	A means many things to monitor and debug. C is one bundle, deployed by copying files; rollback is publishing the previous version.
Scalability	A scales best by design. C scales naturally because each user brings their own computing power, but a very large file is limited by one device.

17 Why Not Microservices, and What It Costs

17.1 Reasons Against Microservices

1. **No scale problem.** Each user processes their own documents on their own device; there is no shared load to split across servers.
2. **It would break our privacy promise.** Processing in a service requires uploading the file (NFR-S1), creating stored personal data we would have to protect.
3. **It would cost money we do not have**, since always-on hosting is not free.
4. **It would slow the people who need speed most.** A blind student relies on instant audio feedback; network round trips and cold starts add delay and silent failures.
5. **The team is too small.** Microservices solve *organisational* problems (many teams); with four members there is only overhead to pay.
6. **The domain is one tightly connected flow.** Open, extract, structure, read and navigate all depend on one document model; splitting it would create fragile calls for no gain.
7. **More parts mean more ways to fail**, while our reliability requirement says a bad file must never leave the user in silence (NFR-R1).

A server-based monolith (Option B) is simpler than A and would be acceptable, but it still needs hosting and sends documents off the device, so it scores below C on privacy, cost and responsiveness with no compensating benefit.

17.2 Keeping the Monolith Modular

A monolith is a good choice only if it stays organised. We therefore set rules that give us the discipline of services without the network:

1. **Layering.** A layer may use layers below it, never above it; information flows upward only through events.
2. **Public interfaces only.** A module is used only through its documented interface.
3. **Adapters at the edge.** Browser features are reached only through adapter classes.
4. **One data owner.** Each kind of stored data is owned by one module.
5. **Replaceable extraction.** The extractor sits behind an interface, so it can later move to a server without changing its callers.
6. **Enforced by tools.** A lint rule and a continuous-integration check fail the build if code imports from a higher layer.

17.3 Consequences and Trade-offs

Consequence	Effect	Mitigation
Good: documents stay on the device	Strong privacy; nothing to secure or delete on a server.	—
Good: zero cost, fast, simple, testable	Free static hosting; direct calls; one build; easy demo.	—
Bad: speed depends on the user's device	A very large file may be slow on an old laptop.	Web Worker; file limits (FR-02); early performance test.
Bad: no sync across devices, no central usage data	Bookmarks stay in one browser; no remote measurement.	Out of scope; feedback through interviews and testing.
Bad: risk of a tangled monolith	Modules could slowly become entangled.	Layer rules, folder layout, automated checks.

18 Evolution Path and Decision Record

18.1 When We Would Revisit the Decision

The decision is right for the current scope, not permanent. Each trigger below would be handled by extracting a *single* service behind an existing interface, not by a wholesale move to microservices.

Trigger	Possible new service	Seam already in the design
Scanned documents (OCR) or heavy parsing	Server-side extraction service	`DocumentExtractor` interface; format-independent document model.
Accounts and cross-device bookmarks	Sync service with a user database	`StorageService` adapter.
Automatic image or equation descriptions	Description service (AI)	Content blocks of kind *notice*.
Usage analytics (with consent)	Telemetry service	Event bus; no module depends on it.

Validation. The decision rests on one assumption: a browser can process a typical course document fast enough (NFR-P4: 20 s for 100 pages). We test it in a performance spike in Increment 1 with real course files. If it fails, we introduce *one* small extraction service and keep everything else unchanged, recorded as a new decision record.

18.2 Architecture Decision Record

ID and title	**ADR-001: Modular monolith running entirely on the client** (accepted as a draft; to be reviewed with the supervisor)
Context	Privacy, instant response, zero cost, and delivery by four members in one semester; each user works alone on their own device.
Decision	One web application bundle containing all modules, running in the browser, with document processing in a Web Worker. No server application, no microservices. Modules kept separate by enforced layer rules.
Review triggers	The performance spike fails; accounts, sync, OCR or shared libraries become requirements; the team or user base grows substantially.

PART V

Object-Oriented Design Model

19 The Design Model and Why We Chose It

19.1 Summary

Following Pressman and Maxim, the software design model has four elements. The table shows where each one lives for Hamnawa.

Design element	What we produced	Where
Data design	Document model; stored records; ERD	Part V, Section 21.2; Part III
Architectural design	Layers, processing pipeline, modular monolith	Parts III and IV
Interface design	Three screen regions, keyboard shortcuts, voice commands, spoken announcements	Requirements FR-26 to FR-47; `UIController`, `Announcer`
Component-level design	**Classes, interfaces, patterns, state and sequence behaviour**	This part

Design model chosen. Object-oriented design, modelled in UML (class, state and sequence diagrams), guided by encapsulation, abstraction, polymorphism, composition over inheritance and the SOLID principles. Design patterns are used only where they solve a real problem.

19.2 Why Object-Oriented Design

1. **The problem is made of objects.** A document has sections; a section has content blocks; a user has bookmarks, a position and settings; the system uses a speech engine. Classes mirror these nouns directly.
2. **Change is expected, and objects hide change.** PDF and PPTX today, other formats later; browser speech today, another engine tomorrow. Behind an interface, a change touches one class (NFR-M1).
3. **State belongs with behaviour.** Reading has state (playing, paused, position, speed). Keeping it in one class (`PlaybackController`) avoids scattered flags and global variables.
4. **Testing without a browser.** Speech, storage and parsing sit behind interfaces, so tests can use fakes: we can test "read the next sentence" without a microphone, a speaker or a real PDF (NFR-M2).

5. **One model for the whole team.** CRC cards, the ERD and use cases already use an object vocabulary; UML continues the same thread, and TypeScript classes and interfaces let the design be coded almost one-to-one.

19.3 Alternatives Considered

Approach	Why it is not the main model
Structured (function-oriented) design	Shared state (position, settings, document) would be passed around or made global; adding a format or engine would change many functions.
Pure functional design	A good fit for stateless text cleaning, where we use it; but reading, speech and focus are stateful and event-driven, which objects model more naturally.
UI-framework components only	Mixes interface and logic, so core behaviour cannot be tested without a browser.
Data-centred design	There is little persistent data and no server; the complexity is in behaviour, not storage.

Pragmatic note. The design is object-oriented, not object-obsessed: stateless helpers such as text-cleaning rules are plain functions; objects are used where there is state, a responsibility to protect, or a change to hide.

20 Concepts and Principles Applied

Concept or principle	How Hamnawa uses it
Encapsulation	`PlaybackController` owns the current position; others call `play()` or `next()`. `BookmarkManager` is the only class that changes bookmarks.
Abstraction and interfaces	`SpeechSynthesizer` says "speak this text"; callers know nothing of browser details. Four small interfaces guard the boundaries.
Inheritance and polymorphism	`Heading`, `Paragraph`, `ListItem` and `Notice` specialise `ContentBlock`. The reader calls `cue()` on every block with no `if` on the block type.
Composition	A `DocumentModel` owns its sections; a section owns its blocks. Closing the document discards them all.
Single responsibility	Each class has one reason to change: `Announcer` only delivers messages; `ResumeService` only remembers positions.

Concept or principle	How Hamnawa uses it
Open/closed	Adding a format means adding one new extractor class; `DocumentLoader` is not edited.
Liskov substitution	Any `SpeechSynthesizer`, real or fake, works wherever one is expected.
Interface segregation	Four small interfaces, not one large "Platform" interface.
Dependency inversion	Controllers depend on interfaces; the concrete adapter is supplied at start-up (dependency injection).
High cohesion, low coupling	Related operations stay together; classes talk through interfaces and events.

21 Static Model (Class View)

21.1 Types of Design Classes

Class type	Hamnawa classes
User interface	`UIController`, `Announcer`
Business domain	`DocumentModel`, `Section`, `ContentBlock` and its subclasses, `Bookmark`, `ReadingState`, `Settings`
Process	`DocumentLoader`, `StructureAnalyzer`, `PlaybackController`, `NavigationController`, `VoiceCommandController`, `CommandParser`, `BookmarkManager`, `ResumeService`, `SettingsManager`
Persistent	`Bookmark`, `ReadingState`, `Settings` (stored through `Storage`)
System	Speech and storage adapters, `EventBus`, document extractors

21.2 Domain Model: the Document

The document model is the heart of the system. It is the same for PDF and PPTX, so everything after extraction (reading, navigation, bookmarks) is format-independent. A section can hold sub-sections, and content blocks specialise a common parent: the *Composite* structure. The notation used in the class diagrams is shown first.



Figure 9: Notation used in the class diagrams.

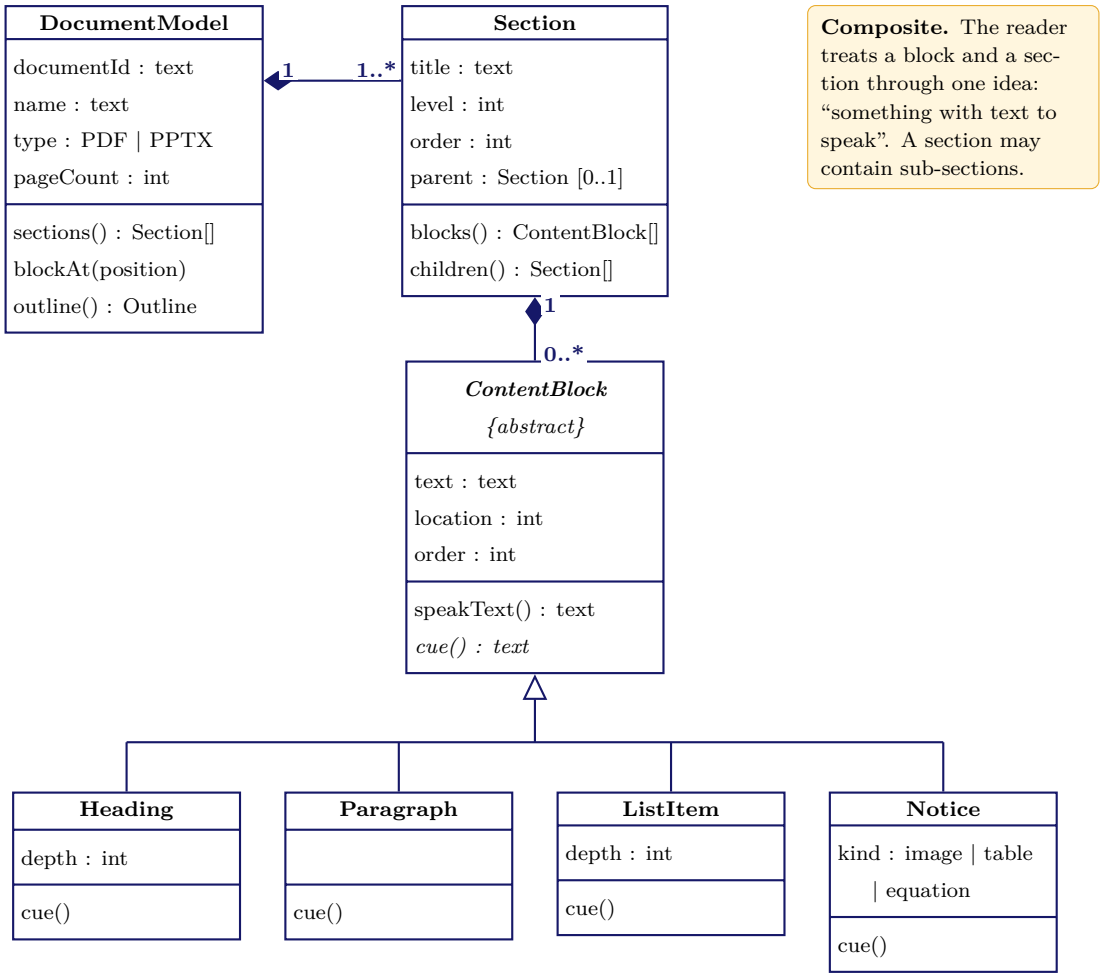


Figure 10: Class diagram of the document model. `cue()` is overridden in each subclass (polymorphism).

Why this shape.

- `DocumentModel` owns its sections, and a section owns its blocks (composition), so there are no dangling parts and memory is released when the document closes.
- `ContentBlock` is abstract. The reader never asks "what kind of block is this?"; it calls `cue()` and gets the right spoken label for headings, lists and notices (FR-24).
- `Notice` stands for images, tables and equations that cannot be read, so the reader announces "image, no description" instead of silently skipping content (FR-16).
- `Bookmark`, `ReadingState` and `Settings` are simple records stored through the `Storage` interface (see the ERD in Part III).

21.3 Interfaces at the Boundaries

Four interfaces separate our logic from everything that can change or fail: the file format, the speech engine, the microphone and storage. The classes on the left depend only on the interface; the classes on the right implement it. In tests, the *fake* replaces the real adapter.

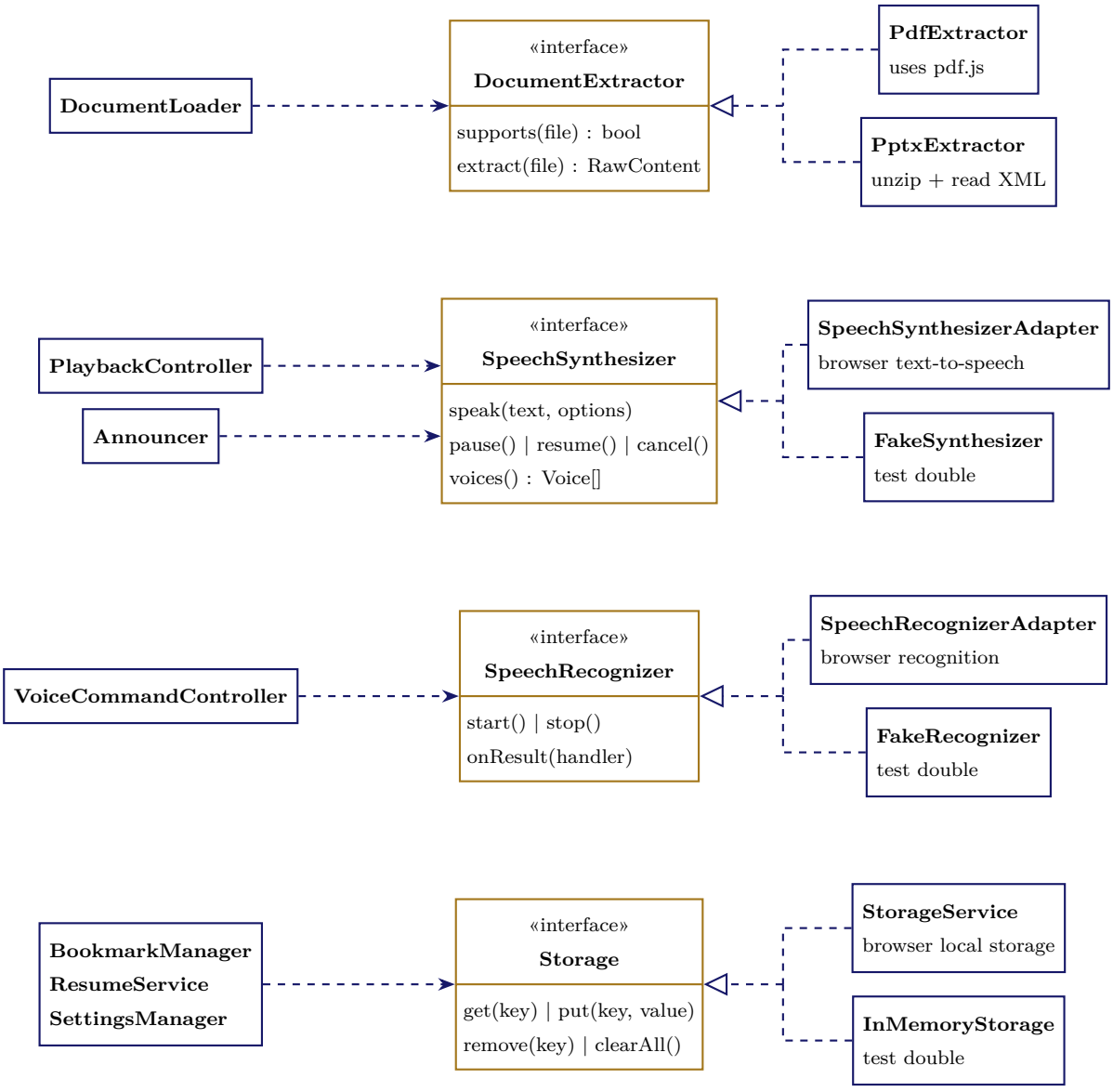


Figure 11: Interfaces at the boundaries. Dashed arrows from left show use; dashed triangles from right show implementation. Controllers never mention a browser feature.

- **Open/closed.** Supporting EPUB later means adding one `EpubExtractor`; `DocumentLoader` chooses the extractor whose `supports(file)` is true (Strategy).
- **Failure containment.** A browser speech failure is caught in `SpeechSynthesizerAdapter` and reported through one error path (NFR-R3).
- **Test doubles.** `FakeSynthesizer` records what would have been spoken, so a test can assert it without audio.

21.4 Class Specifications

The main process classes are specified below. Attributes are private; the operations listed are the public interface. Responsibilities and collaborators are on the CRC cards.

Class	Attributes (private)	Public operations
DocumentLoader (Facade)	extractors; analyzer; limits	`load(file) : DocumentModel`; `cancel()`; publishes *DocumentReady*, *LoadFailed*
StructureAnalyzer	rules : CleaningRule[]	`structure(raw) : DocumentModel`
PlaybackController	state; position; model; synthesizer; settings	`play()`, `pause()`, `resume()`, `stop()`, `repeat()`, `next(unit)`, `previous(unit)`, `setRate(r)`
NavigationController	model; position	`nextSection()`, `goTo(n)`, `openOutline()`, `whereAmI()`, `search(term)`
VoiceCommandController	recognizer; parser; listening : bool	`enable()`, `disable()`; executes the `Command` from the parser
BookmarkManager	bookmarks; storage	`add(label)`, `list()`, `remove(id)`, `jumpTo(id)`
Announcer	queue; synthesizer; liveRegion	`say(text, priority)`, `error(text)`, `progress(percent)`
EventBus	subscribers : Map	`publish(event)`, `subscribe(type, handler)`

21.5 Design Patterns Used

A pattern is used only when it removes a concrete problem.

Pattern	Where	Problem it solves	Requirements
Composite	Document model: sections containing sections and blocks.	Reader and outline treat parts and wholes alike.	FR-15, FR-28 to FR-31

Pattern	Where	Problem it solves	Requirements
Strategy	`DocumentExtractor`: PDF and PPTX extractors.	Choose how to read a file at run time; add formats without editing the loader.	FR-08, FR-09, NFR-M1
Adapter	Speech output, speech input and storage adapters.	Hide browser differences and failures behind our own interface.	NFR-C1 to C3, NFR-R3, FR-39
Facade	`DocumentLoader`.	One call (`load`) hides validate, extract, clean and structure.	FR-02 to FR-05
Observer	`EventBus`.	Lower layers inform upper layers without calling upward.	NFR-M1
Command	Voice and keyboard commands become `Command` objects with `execute()`.	The same action by key and by voice; unknown phrases handled in one place.	FR-35, FR-36
State	`PlaybackController` (Idle, Ready, Playing, Paused).	Operations behave correctly in each state.	FR-18 to FR-22
Dependency injection	Application start-up.	Controllers receive interfaces, not concrete classes.	NFR-M2

22 Dynamic Model (Behaviour)

22.1 Event Flow (Observer)

Information goes down by calls and up by events. A publisher does not know who listens, which keeps the layers independent.

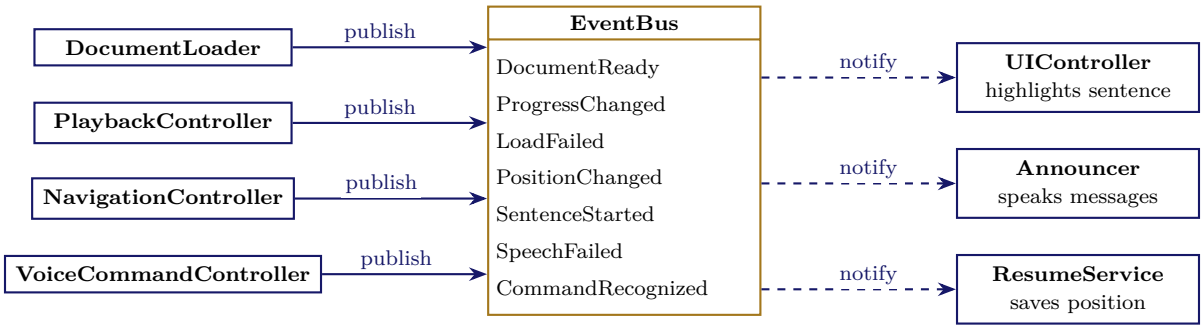


Figure 12: Observer pattern. Controllers publish events; the interface and the services subscribe.

22.2 State of the Reader

Reading has the most state, so it is modelled as a state machine held by `PlaybackController`. Closing a document, from any state, returns the controller to *Idle*.

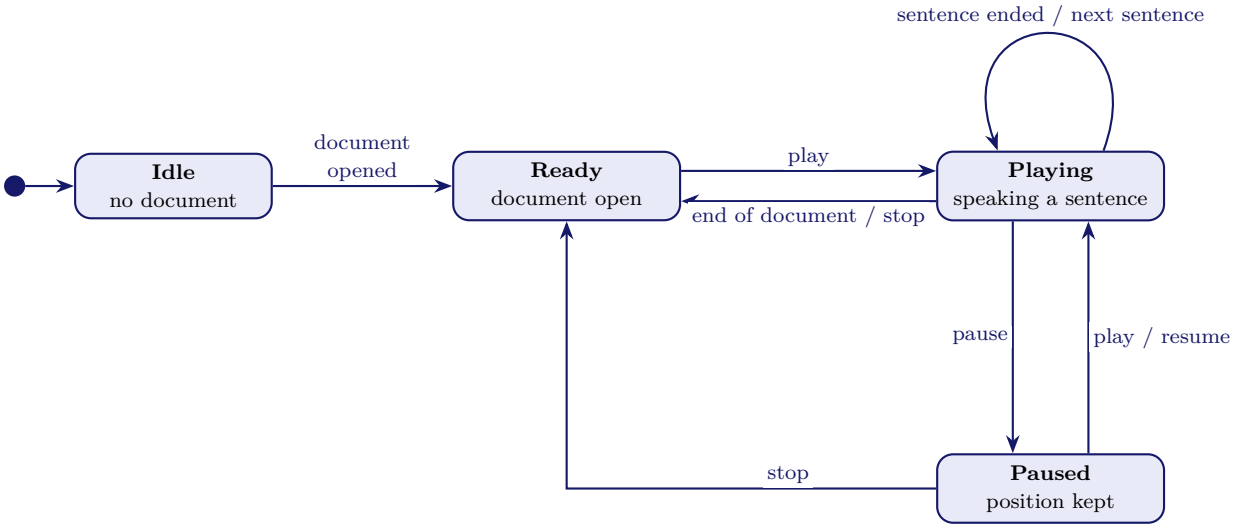


Figure 13: State diagram of `PlaybackController`. A speech failure moves Playing to Paused and announces the problem (NFR-R3).

22.3 Interaction: Open a Document and Read It

The sequence traces use cases UC-01 and UC-02. Read it from top to bottom.

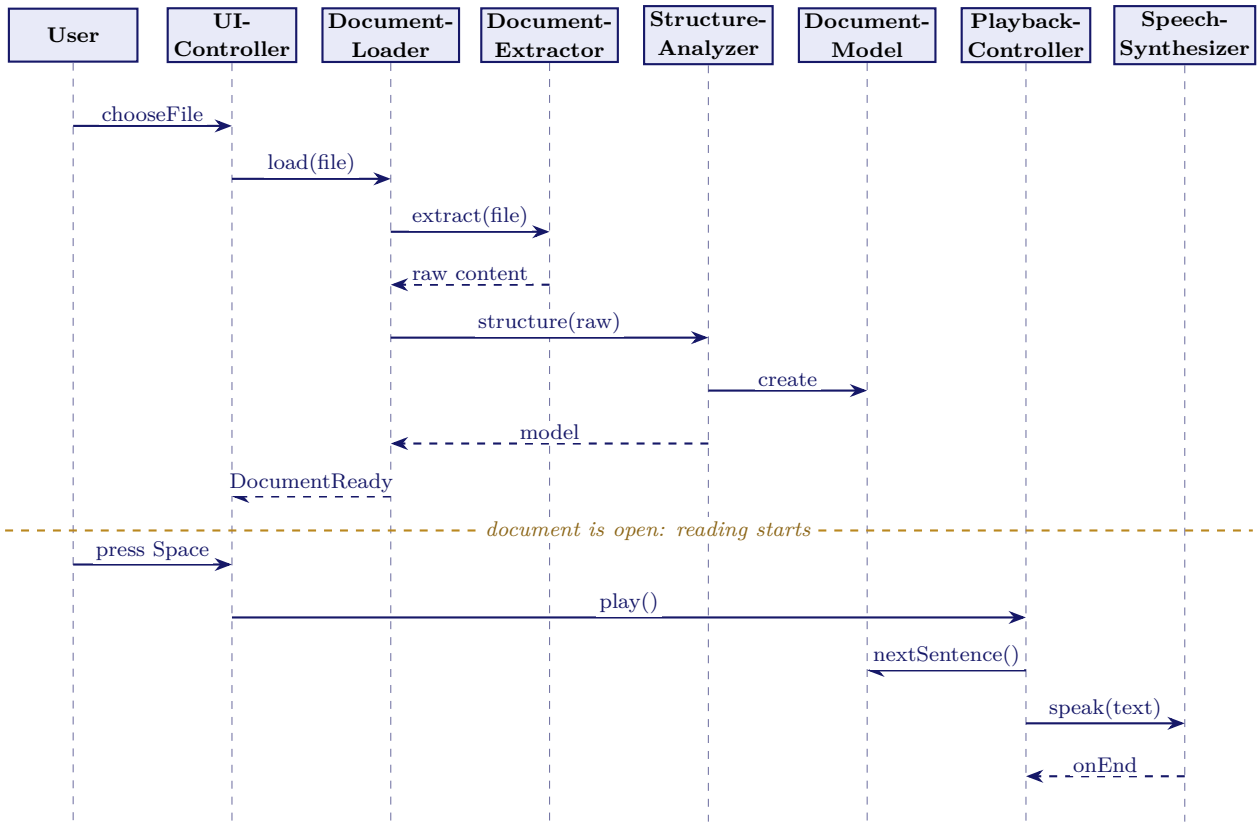


Figure 14: Sequence diagram: opening a document and reading aloud. Solid arrows are calls, dashed arrows are returns or events. Each call goes through an interface, so any participant can be a fake in tests.

Error path. If `extract(file)` fails (corrupt file, scanned PDF without text, file too large), the extractor reports a typed error, `DocumentLoader` publishes *LoadFailed*, and `Announcer` speaks a plain-language message and offers the next step. The user is never left in silence (FR-03, FR-47, NFR-R1).

22.4 From Model to Code and Tests

Each class becomes one source file named after the class, in the folder of its layer; interfaces are separate files. Tests include unit tests for every class using fakes for the four interfaces, a test for each state transition, contract tests that run against both a real adapter and its fake (Liskov), and scenario tests that follow the sequence above on the sample document set.

22.5 Trade-offs

Trade-off	Effect	Mitigation
More classes and interfaces than a quick script	More files and some up-front design work.	Interfaces only at the four boundaries; no speculative classes.
Risk of over-using patterns	Needless complexity for a small team.	A pattern appears only with a stated problem.

Trade-off	Effect	Mitigation
Events are harder to trace than calls	Debugging an event chain is less direct.	Event log in development mode; a short, fixed list of event types.
Model may drift from code	Diagrams go out of date.	Same class names in code and diagrams; update after every increment.

23 Conclusion and Next Steps

This report has defined Hamnawa from the problem to the class model:

- **Why:** a free, structure-aware reader that lets visually impaired students open ordinary course documents and move through them by keyboard and voice (Part I).
- **What:** a prioritised, testable set of requirements, each with an identifier and a measurable fit criterion (Part II).
- **How it is organised:** a four-layer architecture with a document-processing pipeline, adapters at the browser boundary and a format-independent document model (Part III).
- **How it is packaged:** a modular monolith running entirely in the browser, chosen for privacy, zero cost, speed and team fit (Part IV).
- **How it is built:** object-oriented classes and interfaces, with a small set of justified design patterns and fakes for testing (Part V).

No.	Next step	When
1	Interview at least one visually impaired or low-vision student and update requirement priorities.	Increment 1
2	Run the performance spike: parse a 100-page PDF in a Web Worker and measure time and memory (validates ADR-001).	Increment 1
3	Set up the repository, folder structure, lint and layer checks, and the test corpus of 20 course documents.	Increment 1
4	Write the test plan with a traceability matrix from requirements to test cases.	Increment 1
5	Implement the core reader (open, extract, speak) and review the design after each increment.	Increments 1 to 4

Glossary

Screen reader Software that reads on-screen content aloud for blind or low-vision users (for example NVDA).

Text-to-speech Technology that turns written text into spoken audio.

WCAG Web Content Accessibility Guidelines, published by the W3C.

OCR Optical character recognition; reads text from images.

MoSCoW Prioritisation: Must, Should, Could, Won't.

Fit criterion A measurable statement that defines when a requirement is met.

Fingerprint A short value computed from a file's content that identifies it.

Monolith / microservices One deployable program, versus many small independently deployed services.

Web Worker A browser feature that runs code in a background thread.

Adapter A class that hides a browser or library feature behind an interface of our own.

CRC card Class-Responsibility-Collaborator card: a class, what it does, and whom it works with.

ERD Entity-relationship diagram of stored data.

ADR Architecture decision record.

Design pattern A named, reusable solution to a common design problem.

Test double (fake) A simple stand-in for a real component, used in tests.

UML Unified Modeling Language, the standard notation for object-oriented models.

SOLID Five design principles: single responsibility, open/closed, Liskov substitution, interface segregation, dependency inversion.

References

[1] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, McGraw-Hill.

[2] W3C, *Web Content Accessibility Guidelines (WCAG) 2.2*, https://www.w3.org/TR/WCAG22/.

[3] NV Access, *NVDA screen reader*, https://www.nvaccess.org/.